

Report

Part 1

We were asked to develop 3 java classes which are `Dataset`, `StandardizedDataset` and `TestDataset` (which contains the main method to see if the implemented code works as intended). The class `Dataset` contains 2 protected attributes (so that its subclass `StandardizedDataset` can have access to them): `dim`, which is an `int` that represents the dimension of the `Dataset` (i.e. amount of Records it stores); `data`, which is an array of Records. Apart from the constructor method, there are 9 methods: `getDim`, `getData`, `addRecord`, `meanInput` (returns the mean of the inputs), `stdInput` (returns the standard deviation of the inputs), `meanOutput`, `stdOutput`, `standardize`. `StandardizedDataset` inherits from `Dataset` its attributes and methods, but it also has 4 attributes: `mi` (Vector of size `dim` with the means of the inputs), `si` (Vector of size `dim` with the standard deviation of the inputs), `mo` (mean of the outputs), `so` (standard deviation of the outputs). And an additional method `transform`. Finally, `TestDataset`, is the main function which uses the methods of the other classes to check their functionality. It must be mentioned that `standardize`, returns an instance of `StandardizedDataset`, which is obtained by applying the method `transform` (subtracting by the mean and dividing by the standard deviation both the input and output) to each of the records.

Part 2

As we will describe further, we had observed an alternative way of getting elements from the input of `Vector` instance using dot product of the original input vector with a vector that has 1 on the position of wanted element and rest filled with 0. But this approach seemed to us too complicated to be implemented as the code of the method would significantly decrease its readability and would be difficult to understand. Therefore, we decided to use the concept of abstract programming and use a function `getElem` inside `Vector` class which would get us the element of input on the given index, and afterwards, we implemented the function in the `Vector` class as we found it useful not only for mean and standard Deviation, but also for any other request we might have in future.

Moreover, as stated earlier, we use inheritance in the definition of `StandardizedDataset` class from the `Dataset` class to reduce amount of code to be implemented and improve code usability.

In conclusion, so far in this project, we applied such concepts as polymorphism (Lab 1), inheritance, abstraction and encapsulation (applied everywhere as private attributes and protected in `Dataset` class to be able to add new records).

Part 3

When it came to testing the correct implementation of the classes, most of the methods could be tested by checking that the mean and standard deviation of a `StandardizedDataset` are equal to 0 and 1, respectively. That is because the use of the `standardize()` method implies the use of all the methods in both classes, except for the `Dataset` constructor and the getters. Therefore, after testing the remaining methods, if the test proved the correct implementation of `standardize()`, which indeed proved it, we were done.

When implementing the methods for the class `Dataset`, we encountered a difficulty of operating with the elements of the input vectors, as the attribute is private and cannot be unpacked in another class. First of all, for the mean of inputs of each vector we had to sum the values and divide the sum by the dimension of that vector. After some thinking we ended up using the method `dotProduct` with another vector filled with “1”, so as result we get the sum of inputs. Furthermore, for `stdInputs`, we had to get each element of the vector and subtract from that the mean of that vector. For both those operations we had to deal with specific elements of the instance of class `Vector`, which we could have done again with `dotProduct` where there's 1 on needed position and the rest is filled with 0. But this approach was too complex to implement and also the readability of the code would be poor. So, we decided to implement an additional method `getElem` in class `Vector` to subdivide this 1 big problem into smaller steps. For the output there were no difficulties found on our way of implementation.